

CSCE 5300: Introduction to Big Data and Data Science

Chapter 1 (1.2): Big Data Processing Systems



1.2. Big Data Processing Systems



- *The velocity of data generation, and the variety of the structure of data, support for large-scale data analytics constitutes a particularly challenging task.*
- *To address the scalability requirements of today's data analytics, parallel shared-nothing architectures of commodity machines (often consisting of thousands of nodes) have been lately established as the de facto solution.*
- *Various systems have been developed mainly by the industry to support Big Data analysis, including MapReduce, Pregel, Spark, etc.*
- In this section, we give a brief survey of some representative solutions.

1.2. Big Data Processing Systems



- **HADOOP**
- <https://hadoop.apache.org/>



- *Hadoop is an open-source Java implementation of MapReduce, which is a popular programming model for large-scale data processing developed by Google.*
- *It runs on top of a distributed file system called HDFS, which splits input data into multiple blocks of fixed size (typically 64 MB) and replicates each data block several times across computing nodes.*
- Users can submit MapReduce jobs to the Hadoop cluster.

1.2. Big Data Processing Systems



- The Hadoop system breaks each job into multiple map tasks and reduce tasks, with its map tasks computed before the reduce tasks.
- Each map task processes (i.e., scans and records) a data block and produces intermediate results in the form of key-value pairs.
- *Hadoop has evolved to the next generation of Hadoop (i.e., Hadoop MRv2) called YARN, as a large-scale data operating platform and cluster resource management system.*

1.2. Big Data Processing Systems



- *There is a new architecture for YARN, which separates the resource management from the computation model.*
- *Such a separation enables YARN to support a number of diverse data-intensive computing frameworks including Dryad, Giraph, Spark, Storm, and Tez.*
- ***In YARNs architecture, there is a global master named Resource Manager (RM) and a set of per-node slaves called Node Managers (NM), which forms a generic system for managing applications in a distributed manner.***
- The RM is responsible for tracking and arbitrating resources among applications.

1.2. Big Data Processing Systems



- In contrast, the NM has responsibility for launching tasks and monitoring the resource usage per slave node.
- Moreover, there is another component called Application Master, which is a framework-specific entity.
- It is responsible for negotiating resources from the RM and working with the NM to execute and monitor the progress of tasks.

1.2. Big Data Processing Systems



- *Particularly, all resources of YARN are requested in the form of container, which is a logical bundle of resources (e.g., 1 CPUs, 2G memory).*
- As a multitenant platform, YARN organizes user submitted applications into queues and share resources between these queues.
- Users can set their own queues in a configuration file provided by YARN.

1.2. Big Data Processing Systems



- **DRYAD**
- <https://datadryad.org/stash>



DRYAD

- *Dryad is a distributed execution engine that simplifies the process of implementing data-parallel applications to run on a cluster.*
- The original motivation for Dryad was to execute data mining operations efficiently, which has also led to technologies such as MapReduce or Hadoop.
- *Dryad is a general-purpose execution engine and can also be used to implement a wide range of other application types, including time series analysis, image processing, and a variety of scientific computations.*

1.2. Big Data Processing Systems

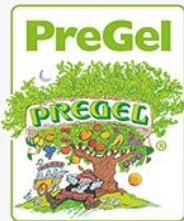


- *The computation is structured as a directed graph: Programs are graph vertices, while the channels are graph edges.*
- *A Dryad job is a graph generator, which can synthesize any directed acyclic graph. These graphs can even change during execution in response to important events in the computation.*
- ***Dryad handles job creation and management, resource management, job monitoring and visualization, fault tolerance, reexecution, scheduling, and accounting.***

1.2. Big Data Processing Systems



- **PREGEL**
- <https://pregelamerica.com/>



- *Pregel is a specialized model for iterative graph applications.*
- *In Pregel, a program runs as a series of coordinated super steps.*
- With each super step, each vertex in the graph runs a user function that can update state associated with the vertex, change the graph topology, and send messages to other vertices for use in the next super step.
- *This model can express many graph algorithms, including shortest paths, bipartite matching, and PageRank.*

1.2. Big Data Processing Systems



- **STORM**

- <https://storm.apache.org/>



- *Storm is a distributed real-time computation system.*
- *In a similar way to how Hadoop provides a set of general primitives for doing batch processing, Storm provides a set of general primitives for doing real-time computation, which greatly eases the writing of parallel real-time computation.*
- *It can be used for processing messages and updating databases (stream processing), performing a continuous query on data streams and streaming the results into clients (continuous computation), parallelizing an intense query like a search query on the fly (distributed Remote Procedure Calls (RPC)), and more.*

1.2. Big Data Processing Systems



- Storm's small set of primitives satisfies a stunning number of user needs.
- *For example, Storm scales to massive numbers of messages per second.*
- To scale a topology, all you have to do is add machines and increase the parallelism settings of the topology.
- *As an example of Storm's scale, one of Storm's initial applications processed 1,000,000 messages per second on a 10-node cluster, including hundreds of database calls per second as part of the topology.*

1.2. Big Data Processing Systems



- **SPARK**

- <https://spark.apache.org/>



- *Spark is a fast in-memory data processing system that achieves high performance for applications through caching data in memory (or disk) for data sharing across computation stages.*
- It is achieved with the *resilient distributed dataset* (RDD) in-memory storage abstraction for computing data, which is a read-only, partitioned collection of records.
- Each RDD is generated from data in stable storage or other RDDs through *transformation* operations such as **map**, **filter**, and *reduceByKey*.

1.2. Big Data Processing Systems



- Notably, the *transformation* is a *lazy* operation that only defines a new RDD without immediately computing it.
- To launch RDD computation, Spark provides another set of *action* operations such as *count*, *collect*, and *save*, which return a value to the application or export data to a storage system.
- **For RDD caching, Spark offers five storage levels, i.e., *memory only*, *memory and disk*, *memory only ser*, *memory and disk ser*, and *disk only*.**
- MEMORY_ONLY_SER – the same as MEMORY_ONLY but it stores RDD. It takes lesser memory (space-efficient) than MEMORY_ONLY as it saves objects as serialized and takes an additional few more CPU cycles to deserialize.

1.2. Big Data Processing Systems



- For a Spark application in execution, the Spark system will spawn a master called *driver*, which is responsible for defining and managing RDDs, and a set of slavers called *executors*, which perform the computations dynamically.
- *The Spark applications can run on either YARN, Mesos, local, or standalone cluster modes.*
- *Some representative Big Data processing systems described in this section in terms of computation model, in-memory computation, resource management type (i.e., CPU, memory, storage, and network), and fairness are summarized in Table 2.1.*

1.2. Big Data Processing Systems



Table 1.2

Comparison of Representative Big Data Processing Systems								
Systems	Computation Model	In-Memory Computation	Resource Management Type				Resource Fairness	
			CPU	Memory	Storage	Network	Single	Multiple
Hadoop	MapReduce	No	Yes	Yes	No	No	Yes	Yes
Dryad	Dryad	No	Yes	No	No	No	Yes	No
Pregel	Pregel	No	Yes	No	No	No	Yes	No
Storm	Storm	No	Yes	No	No	No	Yes	No
Spark	RDD	Yes	Yes	Yes	No	No	Yes	Yes

1.2. Big Data Processing Systems



- There are some other high-level as well as application-specific systems that are built on top of previous data computing systems to form an ecosystem for a variety of applications.
- *For example, for Hadoop, Apache Pig, and Hive are both Structured Query Language (SQL)-like systems that are running on it to support analytical data querying processing.*
- *HBase is a NoSql database system built on top of Hadoop system.*
- *Apache Giraph is an iterative graph processing system running on Hadoop.*

1.2. Big Data Processing Systems



- *Similar to Spark, Shark and Spark SQL are two analytical data query systems built on Spark, while Graphx is a graph processing system for graph applications.*
- **We have also witnessed some other data processing systems/platforms that are running on currently emerging computing devices, such as GPUs.**
- *For example, as an extension of Pregel for GPU platform, a general-purpose programming framework called Medusa has been developed for graph applications.*

1.2. Big Data Processing Systems



- **Recommended Big Data Datasets**

- Popular open data repositories:
 - —*UC Irvine Machine Learning Repository*
 - —*Iris dataset*
 - —*MNIS dataset*
 - —*Kaggle datasets*
 - —*Amazon's AWS datasets*
- Meta portals (they list open data repositories):
 - —*<http://dataportals.org/>*
 - —*<http://opendatamonitor.eu/>*
 - —*<http://quandl.com/>*
- Other pages listing many popular open data repositories:
 - —*Wikipedia's list of Machine Learning datasets*
 - —*Quora.com question*
 - —*Datasets subreddit*